INTERNSHIP REPORT

PFE - Final Year Project

SPRINT 8

Drupal Entity & Field APIs

Full Name : AYOUB BARAHAL

Duration: 1 Week

Status: 100% Completed

"Drupal entity and field APIs provide the foundation for modular, configurable, and maintainable site data."

Contents

1 Entity, Field API & Configuration Entities

Learned core Drupal APIs: Entity API, Field API, and Configuration Entities

Overview

Drupal data is managed through three main systems:

- **Entity API**: create and manage site data objects (users, nodes, taxonomy)
- **Field API**: add and manage fields (text, image, email) in entities
- **Configuration Entities / Configuration API**: store and manage site settings (roles, image styles, module configurations)

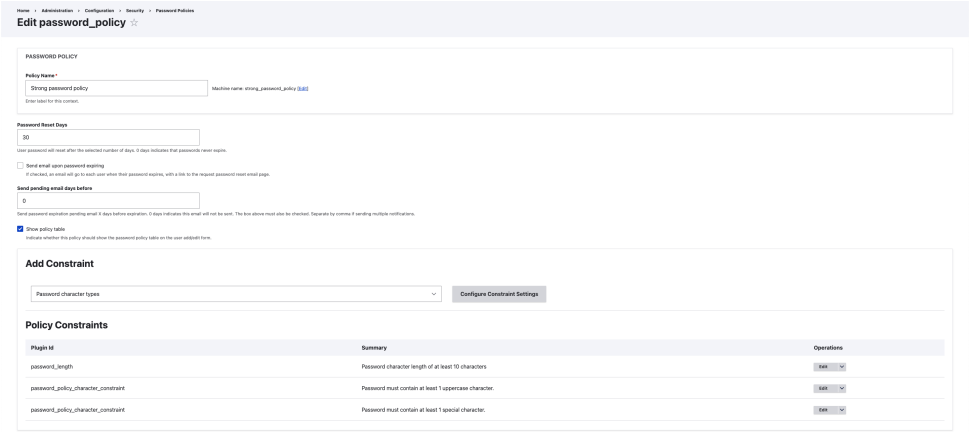
1.1 1. Password Policy Constraint using hook_entity_base_field_info_alter

Added password constraint on user entity pass field using PasswordPolicy module

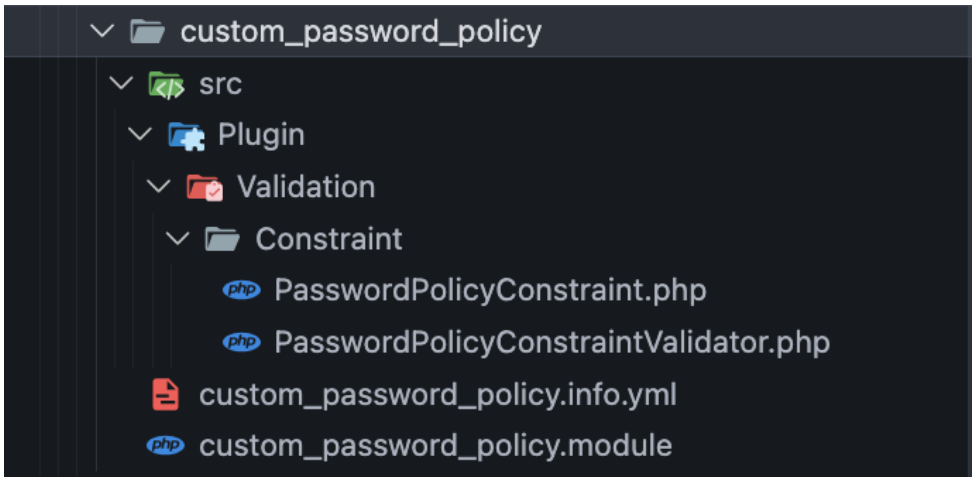
Steps:

1. Install and configure `password_policy` module
2. Implement custom constraint class `PasswordPolicyConstraint` injected with `password_policy.validator` service
3. Alter user base field via `hook_entity_base_field_info_alter`

1.1.1 Install and configure `password_policy` module



1.1.2 Implement custom module custom_password_policy



1.1.3 PasswordPolicyConstraint

```php
<?php
namespace Drupal\custom_password_policy\Plugin\Validation\Constraint;

use Symfony\Component\Validator\Constraint;

/**
 * @Constraint(
 *   id = "PasswordPolicyConstraint",
 *   label = @Translation("Password Policy Constraint")
 * )
 */
0 references | 0 implementations
class PasswordPolicyConstraint extends Constraint {

  0 references
  public $message = 'Password does not meet the policy requirements.';

}
```

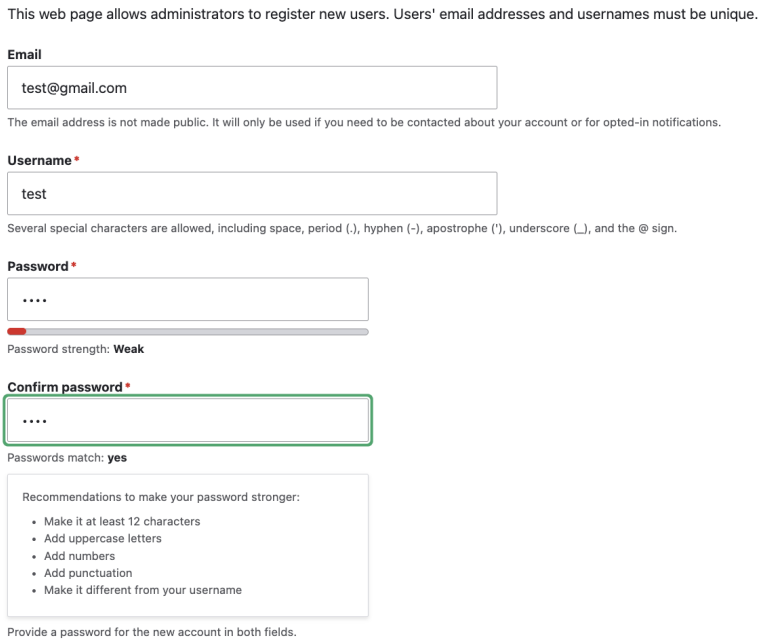
1.1.4 password_policy.validator

```
15 references | 0 overrides
26  public function validate($value, Constraint $constraint): void {
27
28    $account = $this->context->getRoot()->getEntity();
29
30    if ($account instanceof UserInterface) {
31
32      $password = (string) $value->value;
33
34      if (!$this->validator->validatePassword(password: $password, user: $account)) {
35
36        $this->context->buildViolation(message: $constraint->message)
37          ->addViolation();
38
39      }
40
41    }
42
43  }
44
45 }
```

1.1.5 hook_entity_base_field_info_alter

```
0 references
8  function custom_password_policy_entity_base_field_info_alter(&$fields, EntityTypeInterface $entity_type): void {
9
10   if ($entity_type->id() === 'user') {
11
12     if (isset($fields['pass'])) {
13
14       $fields['pass']->addConstraint('PasswordPolicyConstraint');
15
16     }
17
18   }
19
20 }
```

1.1.6 Test



1.2 2. AccessResult in Drupal

Implemented object-oriented access checks with AccessResult

Key Concepts:

- Replaces boolean TRUE/FALSE with richer, expressive results
- Supports conditional helpers: `allowedIf()`, `forbiddenIf()`, `allowedIfHasPermission()`
- Can chain results: `orIf()`, `andIf()`
- Built-in caching: cache contexts, tags, max-age

Example:

```
use Drupal\Core\Access\AccessResult;

public function access(AccountInterface $account) {
  if ($account->hasPermission('administer site')) {
    return AccessResult::allowed();
  }
  return AccessResult::forbidden('User lacks admin permission.');
}

// Conditional helpers
```

```
AccessResult::allowedIf($account->isAuthenticated());
AccessResult::forbiddenIf($node->isPublished() === false);
AccessResult::allowedIfHasPermission($account, 'edit articles');
```

1.3 3. Field Definition Formatters

Retrieved field definitions and created multiple formatters for field types

Get field definition using code:

```
$field_definition = $entity->getFieldDefinition('field_title');
```

Multiple formatters for a field type:

- A field type can have multiple formatters (e.g., ‘text$_{d}efault$‘, ‘$text_{t}rimmed$‘, ‘$text_{s}ummary_{o}r_{t}rimmed$‘)

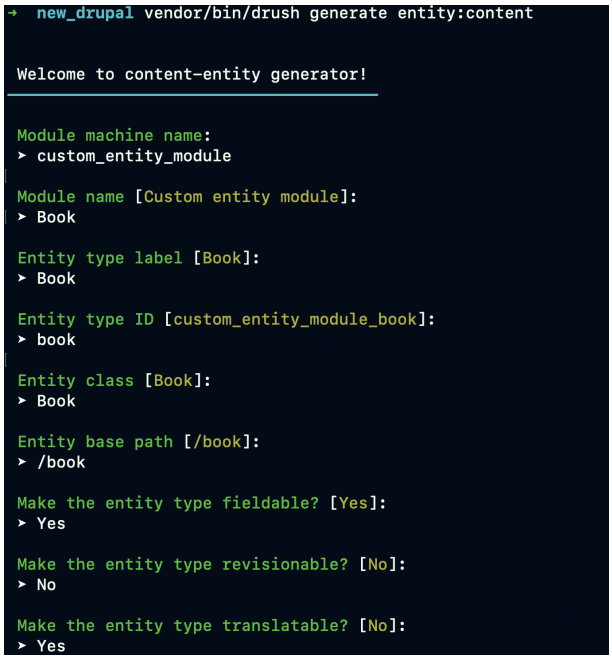
- Each formatter is a separate PHP class with annotation ‘@FieldFormatter‘ and extends ‘FormatterBase‘

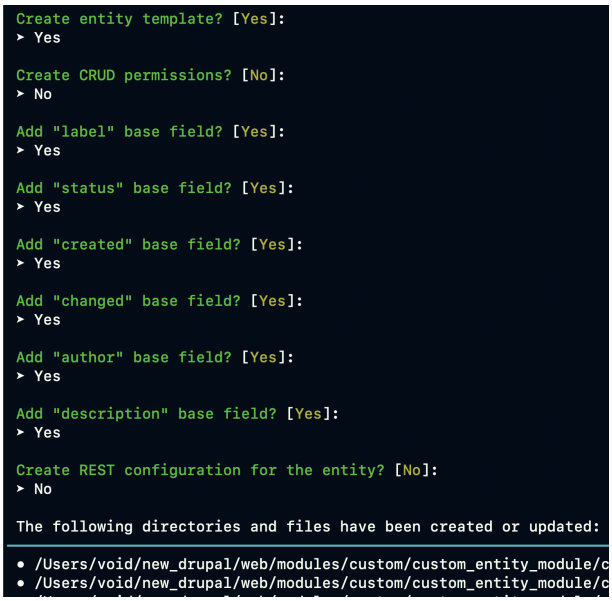
1.4 4. Configuration Retrieval using Drush

Retrieved module configuration via Drush commands

Example:

```
drush cget module_name.settings
```





2 Challenges & Solutions

⚠ **Challenge:** Adding password constraint on user entity
◎ **Solution:** Installed Password Policy module, created custom constraint class, injected validator service, and altered base field definition. Verified via user registration and password form.

⚠ **Challenge:** Understanding AccessResult logic
◎ **Solution:** Studied conditional helpers, chaining, and caching mechanisms. Applied AccessResult in controller access check.

⚠ **Challenge:** Managing multiple field formatters
◎ **Solution:** Created multiple formatters for text fields, tested annotation-based registration, verified display settings in Drupal UI.

3 Technical Achievements

Mastered Drupal Entity API for content and configuration entities

Implemented custom password constraint using PasswordPolicy module

Learned object-oriented AccessResult with caching

Retrieved field definitions and created multiple formatters

Retrieved module configuration via Drush

4 Work with configuration features

4.1 Objective

The goal of this exercise was to set up two Drupal 10 instances (Development and Production) and practice configuration synchronization between them, including the proper handling of development-only modules using config_split.

5 Environment Setup

Two Drupal 10 Instances

- **Development (DEV):** Local environment for development and testing
- **Production (PROD):** Remote server for live site

6 Development Environment Configuration

Modules Installed

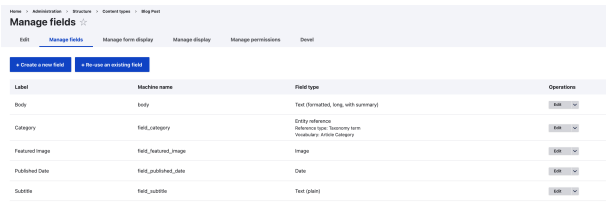
- **devel** — Development utilities and debugging tools
- **devel_generate** — Generate dummy content for testing

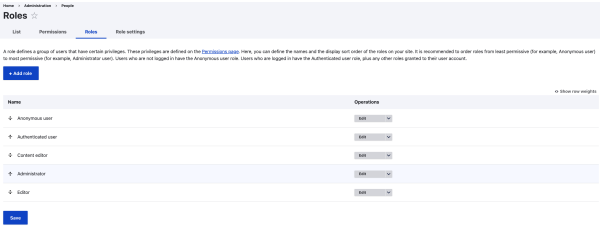
Taxonomy Created — Article Category

Content Type Created — Blog Post

Role Created — Content Editor

- Create, Edit (own & any), Delete Blog Post content
- View published content





7 Config Split — Excluding Devel from Production

Problem

A plain config:export includes ALL enabled modules in core.extension.yml, including devel. This forces devel to be installed on production — which is a security and performance risk.

Solution — config_split Module

1. Install config_split: `composer require drupal/config_split`
2. Create a "dev" split configuration
3. Assign devel and devel_generate to the dev split

Figure 1: split module config

8 Configuration Synchronization

Process

The following steps were performed to synchronize configuration from Dev to Prod:

1. Export configuration from DEV: `drush config:export`
2. Transfer files to PROD
3. Check status on PROD: `drush config:status`
4. Import configuration on PROD: `drush config:import`

```
➜  drupal_dev ./vendor/bin/drush config:export -y

 [success] Configuration successfully exported to ../config/sync.
../config/sync
➜  drupal_dev
```

Figure 2: export config

```
➜  drupal_dev cp -r ~/drupal_dev/config/sync/* ~/drupal_prod/config/sync/

➜  drupal_dev
```

Figure 3: transfer files into prod

```
➜  drupal_prod ./vendor/bin/drush config:import -y

+------------+-----------------------------------------+-----------+
| Collection | Config                                  | Operation |
+------------+-----------------------------------------+-----------+
|            | config_split.config_split.dev_split     | Create    |
|            | core.extension                          | Update    |
|            | core.menu.static_menu_link_overrides    | Update    |
|            | search.settings                         | Update    |
|            | core.entity_view_display.user.user.default | Update    |
|            | core.entity_view_display.user.user.compact | Update    |
+------------+-----------------------------------------+-----------+

 // Import the listed configuration changes?: yes.

 [notice] Synchronized extensions: install devel.
 [notice] Synchronized extensions: install devel_generate.
 [notice] Synchronized configuration: delete devel.settings.
 [notice] Synchronized configuration: create config_split.config_split.dev_split.
 [notice] Synchronized configuration: update core.menu.static_menu_link_overrides.
 [notice] Synchronized configuration: update search.settings.
 [notice] Synchronized configuration: update core.entity_view_display.user.user.default.
 [notice] Synchronized configuration: update core.entity_view_display.user.user.compact.
 [notice] Finalizing configuration synchronization.
 [success] The configuration was imported successfully.
```

Figure 4: import config

9 Work with Cache

9.1 References

- magentaDrupalize.me - Concept of Caching
- magentaDrupal at Your Fingertips - Caching

10 Practice: Create a Custom API Route

10.1 Task Description

- Create a new route `/api/articles`.
- Use `entityQuery` to list 3 article nodes with hardcoded IDs (e.g., 10, 223, 45).
- Ensure at least 10 nodes exist in the site.
- Endpoint should return JSON: `[ {nid: integer, title: string} ]`.
- Test caching behavior with `max-age` and `cache-tags`.

```php
0 references | 0 implementations
class ArticlesController extends ControllerBase
{

    0 references | 0 overrides
    public function getArticles(): CacheableJsonResponse
    {

        $nids = [10, 223, 45];

        $result      = [];
        $cacheTags   = [];

        foreach ($nids as $nid) {
            $node = \Drupal::entityTypeManager()
                ->getStorage('node')
                ->load($nid);

            if ($node) {
                $result[] = [
                    'nid'   => (int) $node->id(),
                    'title' => $node->getTitle(),
                ];

                $cacheTags = array_merge($cacheTags, $node->getCacheTags());
            }
        }

        $response = new CacheableJsonResponse($result);

        $cacheMetadata = new CacheableMetadata();
        $cacheMetadata->setCacheTags($cacheTags);
        $cacheMetadata->setCacheMaxAge(3600); // 1h max

        $response->addCacheableDependency($cacheMetadata);

        return $response;
    }
}
```

Figure 5: postman test

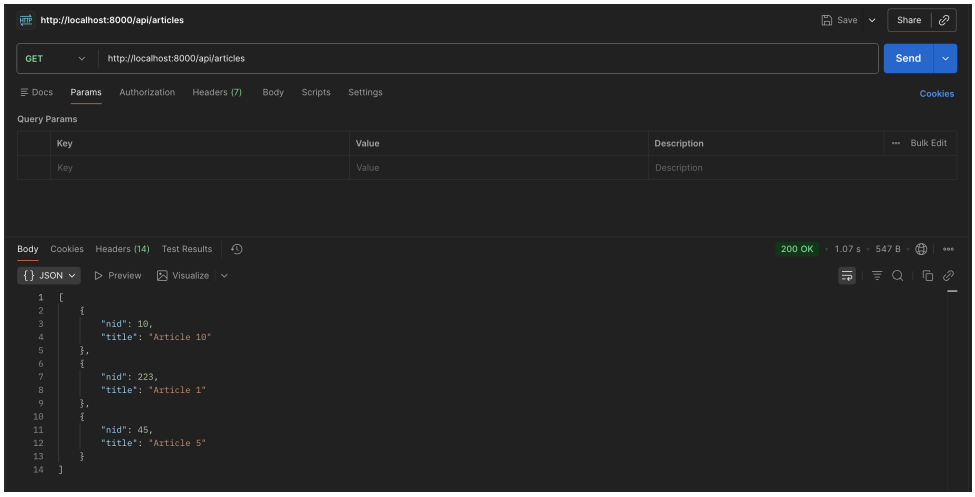


Figure 6: postman test

10.2 Questions Response

10.2.1 Max-Age Cache Test

Modify the title of one of the nodes and query the endpoint again.

Observation: The data is not refreshed instantly. The endpoint continues to return old cached data until cache expires (3600 seconds) or is manually cleared.

10.2.2 Cache-Tags Test

Use cache-tags in the endpoint and repeat the operation.

Observation: When a node with the corresponding cache-tag is modified, Drupal automatically invalidates the cache and the updated data is returned instantly.

11 Work with Data Migration

This sprint focused on working with Drupal data migrations, including using `migration_lookup`, importing from different data sources, rolling back migrations, and processing data fields before import.

12 References

- magentaYouTube Tutorial Part 1
- magentaYouTube Tutorial Part 2
- magentaDrupal 8 Migration: Basic Data (Part 1)
- magentaDrupal 8 Migration: Taxonomy Term References (Part 2)
- magentaDrupal 8 Migration: Files Images (Part 3)

13 Migration Questions and Answers

13.1 Role of `migration_lookup`

The **migration_lookup** plugin is used to link two migrations in Drupal. For example, if tags are imported in one migration and articles in another, articles need the IDs of the previously created tags. Since Drupal generates these IDs automatically, `migration_lookup` allows you to **retrieve the ID of each tag from the first migration** and use it in the second migration, ensuring proper linking.

13.2 Importing from a MySQL Source

The migrate module supports CSV, XML, JSON, and SQL databases. To switch from CSV to MySQL:

- Install the appropriate source module. For MySQL, Drupal core provides `migrate_source_sql`.
- Modify the source section in your migration YAML file:

source: plugin: d7$_{n}odeoracustomSQLpluginkey : legacyDBconnectionkeyinsettings.phpdatabase :$
$driver : mysqlhost : localhostdatabase : my_{o}ld_{d}busername : rootpassword :$
$secret$

13.3 Rolling Back a Migration

You can rollback a migration using Drush:

drush migrate-rollback program$_{d}ata$

13.4 Processing Field Data Before Import

Sometimes you want to transform data before saving it (e.g., truncate a string to 10 characters).

Drupal 7: Modify data in the `prepareRow()` PHP method.

Drupal 8/9/10: Use **process plugins** in the migration YAML file. Plugins can be chained so that output from one plugin is the input of the next.

process: field$_{s}hort_{t}itle : -plugin : callbackcallable : substrsource : Titlesubstr(string, start, length) => substr(value, 0, 10)$